

Almacenes vectoriales en el laboratorio: MongoDB Atlas y embeddings

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 11 de 12 · Notas en español (es-ES)

El servicio RAG práctico usó una capacidad vectorial remota de **MongoDB Atlas** y un cliente de embeddings al estilo **Voyage** (en el aula se oía «OH / OHEI»). Ese emparejamiento es una **demostración de clase**, no la única arquitectura RAG, y no un modelo de embeddings de Anthropic. Claude genera texto; otra cosa tiene que convertir los trozos en vectores.

Por qué el laboratorio no es «la base vectorial de Claude»

Vijay fue explícito: a diferencia de otros laboratorios de modelos, **Claude no trae un modelo de embeddings de primera parte** en el relato de este curso. Los stacks al estilo OpenAI o Amazon a menudo sí. Para CCD Foundation debéis poder decir: elegid un proveedor de embeddings, elegid un almacén vectorial, conservad el bucle recuperar-aumentar-generar. MongoDB+Voyage encajó con una base gratuita en la nube y un cliente que podía embeber y hablar con ese almacén.

NO MEMORICÉIS UN SOLO PROVEEDOR COMO RESPUESTA DE EXAMEN

Si un escenario dice «Postgres pgvector», «Chroma», «FAISS (Meta)», «OpenSearch» o «knowledge base de Bedrock», sigue siendo RAG. Las claves del laboratorio y la UI de Atlas en la carpeta PDF son deberes, no una arquitectura única.

Qué se instaló y configuró

- Cliente Anthropic para generar después de recuperar.
- Un cliente vectorial Voyage/Mongo, autenticado con una clave API guardada como algo tipo `VOYAGE_API_KEY` en `.env` (seguid el PDF del grupo; nunca subáis claves).
- `load_dotenv` para que el proceso del cuaderno abra un endpoint remoto tipo Atlas.
- Una clase envoltorio (dicha como «vector index») que sabía **insertar** documentos, guardar vectores de 1024 dimensiones en la demo y **buscar** con distancia/similitud del coseno.

MongoDB aquí no es «solo un almacén de documentos». El curso usó el producto **vectorial** del proveedor: misma marca, otro patrón de acceso (búsqueda tipo k-NN sobre embeddings).

Embeddings en la demo

Un texto de muestra entró en embed; el cuaderno imprimió los cinco primeros números de un array de **1024 dimensiones**. Esos números no son un significado legible; son coordenadas. Tras embeber todos los trozos de `report.md`, el almacén tenía del orden de **15 registros** en esa ejecución: un corpus de juguete para ver insertar + consultar, no un índice de producción.

Ruta de consulta

1. El usuario pide, por ejemplo, explicar las series de Fourier con sencillez.
2. Esa cadena se embebe con el *mismo* modelo de embeddings que en la indexación (mezclar modelos rompe la geometría).
3. La búsqueda por coseno devuelve k textos de trozo (el laboratorio configuraba «cuántos registros traer»).
4. Esas cadenas se concatenan a la pregunta y se envían a Claude por la API de Anthropic.

Otras arquitecturas que el formador nombró a propósito

Enfoque	Cuándo lo mencionó	Qué seguís teniendo que hacer
MongoDB Atlas + embeddings Voyage	Cuaderno principal	Trocear, embeber, coseno (o métrica configurada), aumentar
Knowledge base de Amazon Bedrock + S3	Vía no-code: cubo, auto-sync, embeddings y troceado en consola, luego boto3	Seguir recuperando y generando; auth con AWS CLI o claves
OpenSearch	Alternativa empresarial de recuperación vectorial	Las mismas tres letras
Chroma / FAISS, en memoria o disco local	Pruebas locales; se sugirió LocalStack en lugar de AWS de pago	Seguís necesitando un modelo de embeddings — en su respuesta reutilizó Voyage incluso en local
Extensiones vectoriales de Postgres	Nombradas en el avance RAG del 24 de agosto	Almacén con forma SQL, la misma matemática

Notas operativas de las preguntas

- Indexar una vez frente a buscar al vuelo: la base vectorial se llena en lote (o con un pipeline de sync); cada pregunta es una búsqueda, no un reescaneo completo del Markdown crudo salvo que lo hayáis diseñado así.
- Cachead embeddings o resultados si importan la latencia y la factura.
- Filtrad con metadatos cuando facultades o inquilinos no deban compartir aciertos.
- Los valores por defecto de similitud pertenecen a APIs tipo `similarity_search`; pasad parámetros si necesitáis otro k o umbral.

Seguridad en la entrega UST

Un clúster Atlas gratuito y una clave Voyage valen para el fichero de muestra de investigación médica. No valen para PHI/PII de cliente. RAG concentra exactamente los documentos que no debíais pegar en Chat. Alojád el almacén en la nube aprobada, cifrad y guardad las claves en un gestor de secretos: el `.env` del cuaderno es un muleta didáctica.

Documentación oficial

- **API Messages**
<https://docs.claude.com/en/api/messages>

Dónde se envía el prompt aumentado tras la búsqueda.

- **Visión general de la API**

<https://docs.claude.com/en/api/overview>

Auth y clientes usados junto al SDK del proveedor vectorial.

- **Soporte de PDF**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Alternativa: enviar documentos a Claude sin vuestra propia base vectorial cuando el tamaño lo permite.

- **Citas**

<https://docs.claude.com/en/docs/build-with-claude/citations>

Fundamentar la generación en texto recuperado o adjunto.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Higiene de claves cuando mucha gente comparte una organización Anthropic.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.